

Laporan Tugas Besar

Arion Compiler

IF2224 Teori Bahasa Formal dan Otomata



Oleh :
khusus 1352____

Daniel Anindito Nugroho 13524002

Ishak Palentino Napitupulu 13524022

Al Farabi 13524086

Ishaq Irfan Farizal 13524094

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I. Teori Singkat

A. Back-End Compiler

Back-end merupakan fase compiler yang bekerja setelah program melewati tahap front-end. Dalam konteks Milestone 4, front-end telah menghasilkan Decorated AST yang valid, sehingga back-end tidak lagi berfokus pada tokenisasi, grammar, maupun pengecekan semantik dasar. Back-end berfokus pada dua hal utama, yaitu sintesis dan komputasi.

Sintesis adalah proses mengonversi Decorated AST menjadi instruksi operasional standar dalam bentuk Intermediate Code. Komputasi adalah proses mengelola memori dan menjalankan instruksi tersebut pada mesin virtual atau Interpreter. Dengan adanya back-end, Arion tidak hanya dapat memvalidasi program, tetapi juga dapat menjalankan program dan menghasilkan output sebenarnya.

B. Decorated AST

Decorated AST merupakan AST yang telah dilengkapi dengan informasi semantik. Informasi ini meliputi tipe data, scope, serta referensi terhadap tabel simbol. Pada Milestone 4, Decorated AST menjadi masukan utama bagi Intermediate Code Generator. Decorated AST diperlukan karena generator tidak cukup hanya mengetahui bentuk struktur program. Generator juga perlu mengetahui makna dari setiap node. Misalnya, ketika ditemukan ekspresi aritmetika, generator perlu mengetahui tipe operand yang terlibat. Ketika ditemukan identifier, generator perlu mengetahui informasi address dari identifier tersebut agar nilai dapat dimuat atau disimpan pada lokasi memori yang tepat.

C. Intermediate Code

Intermediate Code merupakan representasi perantara antara Decorated AST dan proses eksekusi oleh Interpreter. Bentuk ini digunakan untuk meratakan struktur program yang semula berbentuk pohon menjadi daftar instruksi yang linear dan lebih mudah dijalankan. Decorated AST cocok digunakan untuk analisis karena bentuknya hierarkis. Namun, untuk eksekusi, mesin lebih mudah membaca instruksi secara berurutan. Oleh karena itu, Intermediate Code berperan sebagai jembatan antara pemahaman program dan eksekusi program.

D. Three-Address Code dan Flattening

Three-Address Code (TAC) merupakan salah satu bentuk Intermediate Code yang digunakan untuk memecah ekspresi kompleks menjadi instruksi-instruksi sederhana. Setiap instruksi TAC umumnya hanya melibatkan operand dan hasil dalam jumlah terbatas. Tujuan utamanya adalah menyederhanakan ekspresi tingkat tinggi menjadi langkah-langkah kecil yang lebih mudah dieksekusi. Proses mengubah Decorated AST menjadi Intermediate Code disebut flattening, yaitu meratakan struktur pohon menjadi daftar instruksi linear. Misalnya, ekspresi yang memiliki banyak operator tidak dieksekusi sekaligus, tetapi dipecah menjadi operasi-operasi dasar sesuai urutan evaluasi. Pada implementasi Arion, bentuk Intermediate Code yang digunakan mengikuti konvensi instruksi pada spesifikasi, yaitu instruksi yang dijalankan oleh Interpreter dengan pendekatan Stack Machine. Oleh karena itu, konsep TAC tetap digunakan sebagai

dasar perataan program, sementara eksekusinya dilakukan melalui instruksi berbasis stack.

E. Daftar Instruksi Intermediate Code

Daftar instruksi merupakan kumpulan instruksi yang dikenali oleh Interpreter. Setiap instruksi memiliki operasi tertentu yang akan dijalankan ketika proses eksekusi berlangsung. Dalam spesifikasi Milestone 4, instruksi utama yang digunakan adalah LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, dan RET. Instruksi LIT atau Load Literal digunakan untuk memasukkan nilai literal ke dalam stack. Instruksi LOD atau Load Value digunakan untuk memuat nilai dari suatu address. Instruksi STO atau Store Value digunakan untuk menyimpan nilai ke suatu address. Instruksi INT atau Initiate Memory digunakan untuk membuat memori dengan ukuran tertentu. Instruksi CAL digunakan untuk memanggil fungsi atau prosedur pada baris tertentu, sedangkan RET digunakan untuk keluar dari fungsi atau prosedur. Untuk alur eksekusi, digunakan JMP sebagai Unconditional Jump dan JPC sebagai Conditional Jump. Instruksi OPR digunakan untuk menjalankan operasi tertentu, termasuk operasi bawaan Arion.

F. Instruksi OPR

Instruksi OPR merupakan instruksi yang digunakan untuk menjalankan operasi pada nilai yang berada di stack. Operasi ini mencakup operasi aritmetika, operasi perbandingan, dan operasi output. Operasi aritmetika terdiri atas NEG, ADD, SUB, MUL, DIV, dan MOD. Operasi-operasi tersebut bekerja dengan mengambil nilai dari stack, melakukan perhitungan, kemudian menyimpan hasilnya kembali ke stack. Untuk operasi perbandingan, digunakan EQL, NEQ, LSS, GEQ, GTR, dan LEQ. Hasil dari operasi perbandingan digunakan sebagai kondisi pada percabangan atau perulangan. Selain itu, terdapat operasi WRT dan WRTLN. Operasi WRT digunakan untuk menulis output yang sudah dimuat, sedangkan WRTLN digunakan untuk menulis output dan menambahkan newline. Dalam implementasi Arion, operasi ini menjadi mekanisme utama untuk menghasilkan output program dari Intermediate Code.

G. Intermediate Code Generation

Intermediate Code Generation merupakan proses mengubah Decorated AST menjadi daftar instruksi Intermediate Code. Spesifikasi Milestone 4 menyebutkan bahwa pendekatan yang digunakan bersifat Depth First Search based approach, yaitu menggunakan prinsip penelusuran mendalam pada struktur AST meskipun bukan DFS murni. Pada ekspresi, generator terlebih dahulu menghasilkan instruksi untuk operand, lalu menghasilkan instruksi operasi. Misalnya, untuk operasi penjumlahan, nilai operand dimasukkan ke stack terlebih dahulu, kemudian operasi ADD dijalankan melalui instruksi OPR. Untuk assignment, nilai ekspresi dihasilkan terlebih dahulu, lalu disimpan ke address variabel menggunakan STO. Untuk pemanggilan output seperti writeln, generator menghasilkan instruksi untuk memuat nilai yang akan ditulis, kemudian menjalankan OPR WRTLN. Dengan cara ini, struktur program tingkat tinggi diterjemahkan menjadi urutan instruksi sederhana yang dapat dijalankan oleh Stack Machine.

H. Translasi Control Flow

Control Flow merupakan bagian program yang mengatur percabangan dan perulangan. Struktur seperti if, while, repeat, dan for tidak dijalankan oleh Interpreter

sebagai blok bersarang. Struktur tersebut diratakan menjadi instruksi linear dengan bantuan Label dan Jump.

Pada konsep TAC, Label digunakan sebagai penanda posisi instruksi tertentu. Jump digunakan untuk memindahkan alur eksekusi ke Label tersebut. Spec Milestone 4 menjelaskan penggunaan Jump Bersyarat seperti IF_FALSE ... GOTO ... dan Jump Mutlak seperti GOTO. Dalam instruksi Arion, konsep tersebut direpresentasikan menggunakan JPC dan JMP. JPC digunakan untuk melompat ketika kondisi tidak terpenuhi, sedangkan JMP digunakan untuk melompat tanpa syarat. Pendekatan ini memanfaatkan konsep fall-through, yaitu ketika kondisi benar, eksekusi dapat langsung berlanjut ke instruksi berikutnya tanpa perlu lompatan tambahan.

I. Interpreter dan Virtual Machine

Interpreter merupakan komponen yang menjalankan Intermediate Code. Dalam spesifikasi Milestone 4, Interpreter bertindak sebagai Virtual Machine (VM). VM ini tidak berbentuk perangkat keras, tetapi memiliki logika eksekusi yang menyerupai mesin sederhana.

Interpreter membaca Intermediate Code baris demi baris. Untuk mengetahui instruksi mana yang sedang dijalankan, Interpreter menggunakan Instruction Pointer (IP) atau Program Counter (PC). Pada kondisi normal, IP bergerak ke instruksi berikutnya setelah sebuah instruksi selesai dieksekusi. Namun, pada instruksi JMP atau JPC, IP dapat berpindah ke target tertentu.

J. Siklus Fetch-Decode-Execute

Interpreter bekerja dengan siklus eksekusi yang terdiri atas tiga tahap, yaitu Fetch, Decode, dan Execute. Fetch adalah tahap membaca instruksi yang sedang ditunjuk oleh Instruction Pointer. Decode adalah tahap membedah instruksi untuk mengetahui operasi yang harus dilakukan. Execute adalah tahap menjalankan aksi sesuai instruksi tersebut.

Sebagai contoh, ketika instruksi yang dibaca adalah LIT, Interpreter memasukkan nilai literal ke stack. Ketika instruksi yang dibaca adalah LOD, Interpreter mengambil nilai dari address tertentu. Ketika instruksi yang dibaca adalah STO, Interpreter menyimpan nilai ke address tertentu. Siklus ini terus berjalan hingga program mencapai instruksi RET atau terjadi Runtime Error. Dengan siklus ini, eksekusi program dapat dilakukan secara sistematis dan terkontrol.

K. The Stack dan Stack Frame

The Stack merupakan struktur memori utama dalam arsitektur Interpreter. Stack bekerja dengan prinsip Last In, First Out, yaitu nilai terakhir yang dimasukkan akan menjadi nilai pertama yang diambil. Struktur ini digunakan untuk menyimpan operand sementara, hasil operasi, serta informasi memori eksekusi.

Dalam spesifikasi Milestone 4, memori eksekusi dikelola menggunakan Stack Frame. Stack Frame merupakan bagian memori yang digunakan untuk menyimpan variabel lokal, argumen, serta return address. Setiap Stack Frame memiliki bagian khusus berupa Static link, Dynamic link, dan Return address. Sisanya digunakan untuk menyimpan variabel.

Static link berhubungan dengan scope, Dynamic link berhubungan dengan hubungan pemanggilan saat runtime, dan Return address digunakan untuk menentukan lokasi eksekusi setelah suatu fungsi atau prosedur selesai. Dalam implementasi Arion, struktur ini juga digunakan untuk menjaga agar slot khusus frame tidak tertimpa oleh penyimpanan variabel biasa.

L. Stack Machine

Stack Machine merupakan model eksekusi yang menggunakan stack sebagai pusat operasi. Pada model ini, banyak instruksi tidak membawa semua operand secara eksplisit. Operand dimuat ke stack terlebih dahulu, kemudian operasi mengambil nilai dari stack untuk diproses.

Sebagai contoh, untuk menghitung suatu operasi aritmatika, nilai pertama dan nilai kedua dimasukkan ke stack menggunakan LIT atau LOD. Setelah itu, instruksi OPR dijalankan untuk mengambil nilai-nilai tersebut, melakukan operasi, dan meletakkan hasilnya kembali ke stack. Untuk assignment, hasil yang berada di stack kemudian disimpan ke address variabel menggunakan STO.

M. Runtime Type Checking

Runtime Type Checking merupakan proses pemeriksaan tipe data saat program sedang berjalan. Spesifikasi Milestone 4 menjelaskan bahwa analisis semantik melakukan pengecekan secara statis, sedangkan Interpreter berhadapan dengan nilai aktual saat runtime.

Dalam bahasa yang weakly-typed, Runtime Type Checking menjadi penting karena nilai aktual dapat memicu ambiguitas atau kesalahan saat dieksekusi. Namun, spesifikasi juga menyatakan bahwa Arion merupakan bahasa strongly-typed, sehingga Runtime Type Checking tidak diwajibkan secara penuh.

N. Vulnerabilities pada Interpreter

Interpreter perlu memiliki mekanisme perlindungan agar tidak crash ketika menghadapi instruksi yang salah atau kondisi runtime yang tidak valid. Spec Milestone 4 menyebutkan beberapa bentuk vulnerabilities yang dapat terjadi pada Interpreter.

Kerentanan pada stack meliputi Stack Overflow, Stack Underflow, Stack Smashing, dan Stack Corruption. Stack Overflow terjadi ketika Stack Frame terus ditambahkan hingga melewati batas memori. Stack Underflow terjadi ketika Interpreter mencoba mengambil data dari stack yang kosong. Stack Smashing terjadi ketika penulisan data melewati batas variabel lokal hingga dapat menimpa return address. Stack Corruption terjadi ketika struktur push dan pop tidak seimbang sehingga isi stack menjadi tidak konsisten.

Selain itu, terdapat Out-of-Bounds Variable Access, yaitu akses ke indeks atau address di luar batas yang valid. Terdapat juga Invalid Jump Targets, yaitu kondisi ketika Interpreter diperintahkan melompat ke target instruksi yang tidak ada. Pada operasi aritmatika, Numerical Overflow/Underflow juga dapat terjadi ketika nilai melewati batas representasi numerik. Dalam implementasi Arion, beberapa mekanisme error handling seperti stack overflow, stack underflow, invalid memory access, invalid jump target, dan division by zero digunakan agar program berhenti dengan pesan diagnostic, bukan crash.

II. Perancangan & Implementasi

A. Alur Program

Alur program pada Milestone 4 sengaja dipisahkan dari alur Milestone 1 sampai 3. Milestone 1 sampai 3 masih memakai source code Arion sebagai input, sedangkan Milestone 4 memakai Decorated AST sebagai input backend. Pemisahan ini penting karena kontrak M4 tidak meminta program melakukan tokenisasi dan parsing ulang terhadap source code. Alur M1-M3 adalah sebagai berikut: Source Code Arion -> Lexer -> Parser -> AST Builder -> Semantic Analyzer -> Decorated AST. Alur M4 adalah sebagai berikut: Decorated AST -> DAST Loader -> Semantic Metadata Rehydration -> Intermediate Code Generator -> Interpreter -> Output

Secara lengkap, program terlebih dahulu mengklasifikasikan lokasi file input. Jika input berada di test/milestone-4/, file diperlakukan sebagai Decorated AST dan langsung diarahkan ke jalur backend M4 tanpa melewati lexer, parser, AST builder source, maupun printer semantic report. File DAST kemudian dibaca oleh DecoratedAstLoader, baik dalam bentuk clean Decorated AST maupun dengan header opsional `=== Decorated AST ===`. Loader membaca prefix tree ASCII seperti `|--` dan `--`, membentuk kembali relasi parent-child antar node AST, serta memvalidasi kind node dan field penting seperti nama node, nilai literal, operator, literal kind, tipe, dan tab index bila tersedia. Jika input terlihat seperti source code Arion, node tidak dikenal, atau indentasi tree tidak valid, proses dihentikan dengan diagnostic backend input error.

Setelah AST berhasil dimuat, AST tersebut dianalisis ulang oleh SemanticAnalyzer untuk membangun kembali metadata yang dibutuhkan backend, termasuk symbol table, type registry, lexical level, tab index, block index, dan address variabel. Hasil rehydration ini kemudian diberikan ke IntermediateCodeGenerator. Generator menelusuri Decorated AST secara DFS-based dan menghasilkan instruksi stack machine. Assignment diterjemahkan menjadi pola LIT/LOD lalu STO, percabangan dan perulangan menggunakan JPC serta JMP, sedangkan output menggunakan OPR WRT atau OPR WRTLN.

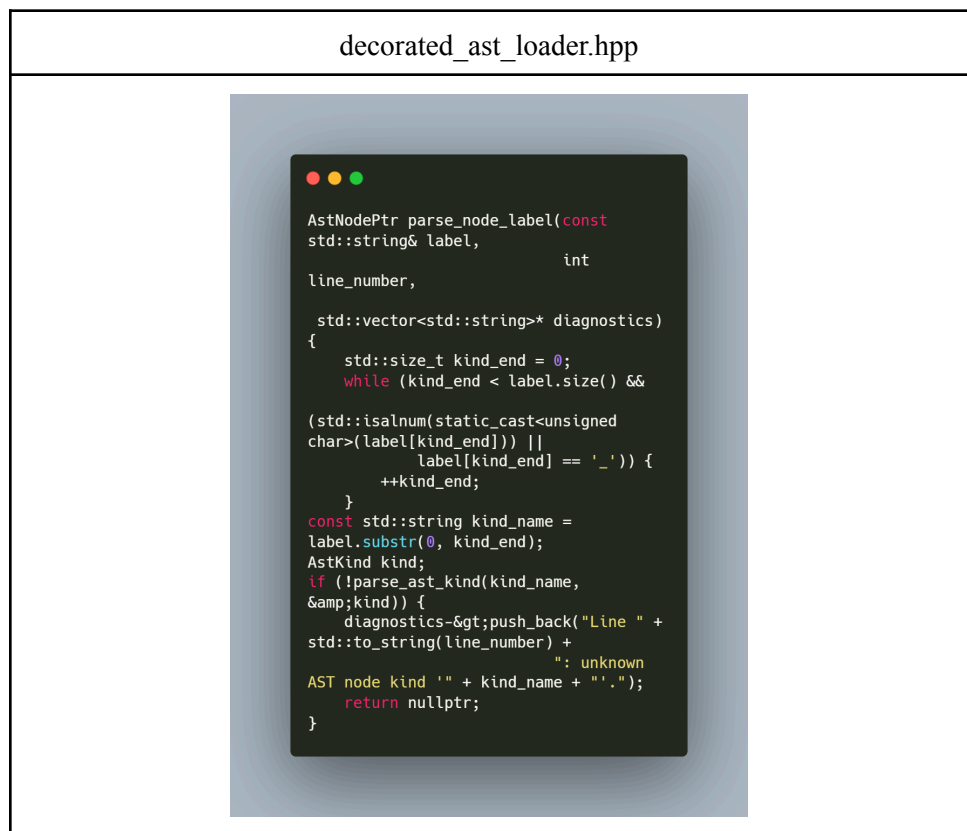
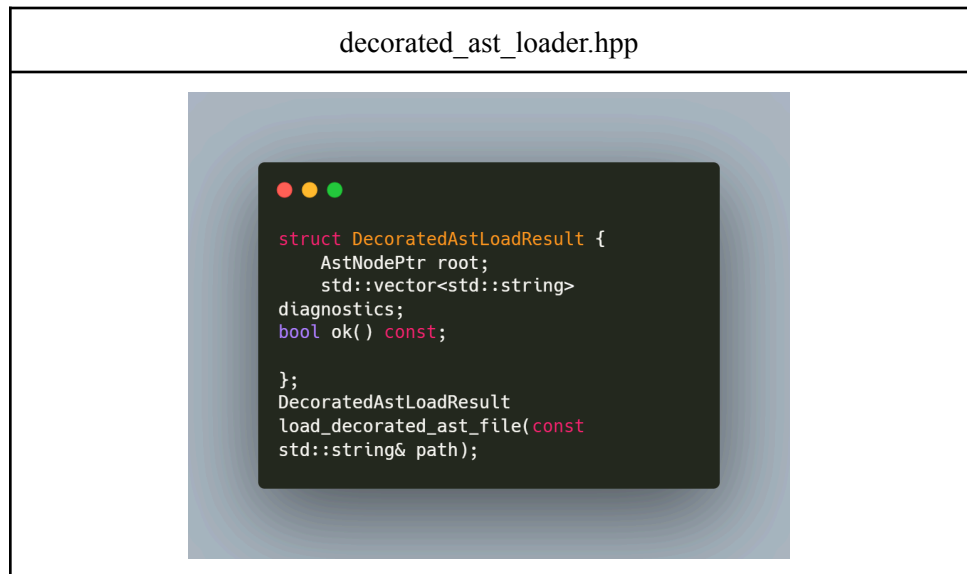
Daftar instruksi yang dihasilkan dijalankan oleh Interpreter. Interpreter memakai runtime stack dan stack frame untuk menyimpan operand, variabel, static link, dynamic link, dan return address. Siklus eksekusi yang digunakan adalah fetch-decode-execute sampai mencapai RET atau diagnostic runtime. Hasil akhir M4 ditulis ke file `_BACKEND.txt` dengan isi minimal sesuai spesifikasi, yaitu section `=== INTERMEDIATE CODE` = berisi instruksi bernomor dan section `= OUTPUT ===` berisi output aktual program.

B. Implementasi

1. Loader Decorated AST

Loader DAST membaca format tree ASCII dari printer semantic. Loader menerima header opsional `=== Decorated AST ===`, berhenti ketika

menemukan section report berikutnya, dan mengekstrak kind node, nama, value, op, literal, serta anotasi tipe yang tersedia. Jika node tidak dikenali atau struktur pohon rusak, loader mengembalikan diagnostic



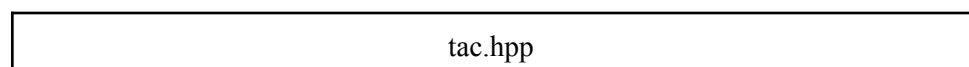
2. Rehydration Metadata Semantic

Setelah AST berhasil dimuat, program menjalankan kembali SemanticAnalyzer pada AST tersebut. Tahap ini membangun ulang decorated_ast, SymbolTable, dan TypeRegistry dari deklarasi yang ada pada DAST, sehingga backend tidak memakai metadata hardcoded.



3. Representasi Intermediate Code

Intermediate Code direpresentasikan sebagai daftar Instruction. Opcode wajib spesifikasi disediakan bersama opcode tambahan yang dibutuhkan implementasi, seperti LDA, LDI, STI, CHK, DUP, dan POP. Operasi aritmetika, relasional, dan output direpresentasikan melalui OPR dengan kode operasi OprCode.



```

enum class OpCode {
    Lit, Lod, Sto, Lda, Ldi, Sti,
    Cal, Int, Jmp, Jpc, Opr, Ret,
    Chk, Dup, Pop
};

enum class OprCode {
    Neg = 1, Add = 2, Sub = 3, Mul = 4, Div
    = 5, Mod = 6,
    Eql = 7, Neq = 8, Lss = 9, Geq = 10,
    Gtr = 11, Leq = 12,
    Wrt = 13, Wrtln = 14
};

struct Instruction {
    OpCode opcode = OpCode::Ret;
    int level = 0;
    int argument = 0;
    RuntimeValue literal_value;
    bool has_literal_value = false;
    std::string comment;
};

```

4. Intermediate Code Generator

Generator melakukan traversal berbasis DFS terhadap Decorated AST. Pada program utama, generator mengalokasikan frame global dengan INT, mengumpulkan deklarasi subprogram bila ada, lalu menghasilkan instruksi untuk statement. Untuk program tanpa subprogram, generator tidak menambahkan JMP main awal yang tidak diperlukan.

code_generator.cpp

```

void generate_program(const
semantic::AstNodePtr& node) {
    current_level = node-
>annotation.lexical_level;
    std::size_t frame_slots =
kFrameHeaderSlots;
    ...
    emit_instruction(Opcode::Int, 0,
static_cast<int>&(frame_slots),
"global frame");

    for (const auto& child : node-
>children) {
        if (child && child->kind
== semantic::AstKind::DeclarationPart)
        {
            collect_subprograms(child);

            generate_declaration_part(child);
        }
    }

    const bool has_subprograms =
!subprogram_order.empty();
    if (has_subprograms) {
        main_jump =
emit_simple(Opcode::Jmp, 0, "main");
    }
}

```

Assignment diterjemahkan dengan menghasilkan nilai ekspresi terlebih dahulu, kemudian menyimpannya ke slot variabel menggunakan STO. Jika target berupa akses array atau record, generator memakai alamat indirect melalui STI.

code_generator.cpp

```

void generate_assignment(const
semantic::AstNodePtr& node) {
    if (!node || node->children.size()
< 2) {
        add_diagnostic(node,
"Assignment statement is missing target
or expression.");
        return;
    }
    const auto& target = node-
&gt;children[0];
    if (!target || target->kind !=
semantic::AstKind::Variable) {
        add_diagnostic(target, "Assignment
target must be a variable.");
        return;
    }

    if (!target->children.empty()) {
        emit_variable_address(target);
        emit_expression(node-
&gt;children[1]);
        emit_simple(OpCode::Sti, 0, "store
indirect");
        return;
    }
}

```

Ekspresi diproses oleh dispatcher emit_expression. Literal, variable, unary operation, binary operation, dan function call diarahkan ke emitter masing-masing. Jika DAST memuat expression node yang belum didukung, generator menghasilkan diagnostic.

code_generator.cpp

```

bool emit_expression(const
semantic::AstNodePtr& node) {
    if (!node) {
        return false;
    }
    switch (node->kind) {
        case semantic::AstKind::Literal:
            return
emit_literal_expression(node);
        case semantic::AstKind::Variable:
            return
emit_variable_load(node);
        case semantic::AstKind::UnaryOp:
            return
emit_unary_expression(node);
        case semantic::AstKind::BinaryOp:
            return
emit_binary_expression(node);
        case semantic::AstKind::Call:
            return emit_user_call(node,
true, false);
        default:
            add_diagnostic(node,
"Unsupported expression node " +
semantic::ast_kind_name(node->kind)
+ ".");
            return false;
    }
}

```

5. Control Flow

Struktur percabangan dan perulangan diratakan menjadi instruksi linear dengan target numerik. Generator menyimpan indeks instruksi JPC atau JMP sementara, lalu melakukan patch setelah alamat akhir blok diketahui.

code_generator.cpp


```

void generate_if(const
semantic::AstNodePtr& node) {
    emit_expression(node->children[0]);
    const std::size_t jpc_line =
    emit_simple(OpCode::Jpc, 0, "if
false");
    generate_statement(node-
>children[1]);
    ...
    patch(jpc_line, current_line());
}
void generate_while(const
semantic::AstNodePtr& node) {
    const std::size_t loop_start =
    current_line();
    emit_expression(node->children[0]);
    const std::size_t jpc_end_line =
    emit_simple(OpCode::Jpc, 0, "while
end");
    generate_statement(node->children[1]);
    emit_simple(OpCode::Jmp,
    static_cast<int>(loop_start), "while
start");
    patch(jpc_end_line, current_line());
}

```

for dan case memakai prinsip yang sama. Pada for, generator membuat inisialisasi counter, pengecekan batas, increment atau decrement, lalu lompatan kembali ke awal loop. Pada case, selector disimpan di stack, dibandingkan dengan setiap label menggunakan DUP dan OPR EQL, kemudian cabang yang tidak cocok dilewati menggunakan JPC.

code_generator.cpp

```

void generate_for(const
semantic::AstNodePtr& node) {
    ...
    emit_expression(node->children[0]);
    emit_instruction(Opcode::Sto,
lexical_level, address, entry-
>identifier);
    const bool downto = normalize(node-
&gt;op) == "downtosy";
    const std::size_t loop_start =
current_line();
    emit_instruction(Opcode::Lod,
lexical_level, address, entry-
&gt;identifier);
    emit_expression(node-&gt;children[1]);
    emit_opr(downto ? Opcode::Geq :
Opcode::Leq);
    const std::size_t jpc_end_line =
emit_simple(Opcode::Jpc, 0, "for end");

    generate_statement(node-
&gt;children[2]);
    emit_instruction(Opcode::Lod,
lexical_level, address, entry-
&gt;identifier);
    emit_literal(RuntimeValue::integer(1),
"for step");
    emit_opr(downto ? Opcode::Sub :
Opcode::Add);
    emit_instruction(Opcode::Sto,
lexical_level, address, entry-
&gt;identifier);
    emit_simple(Opcode::Jmp,
static_cast<int>(loop_start),
"for start");
    patch(jpc_end_line, current_line());
}

void generate_case(const
semantic::AstNodePtr& node) {
    emit_expression(node-
>children.front());
    std::vector<std::size_t> end_jumps;
    for (std::size_t i = 1; i < node-
&gt;children.size(); ++i) {
        ...
        emit_simple(Opcode::Dup, 0, "case
selector");
        emit_expression(label);
        emit_opr(Opcode::Eql);
        const std::size_t no_match =
emit_simple(Opcode::Jpc, 0, "case no
match");
        ...

        end_jumps.push_back(emit_simple(Opcode
::Jmp, 0, "case end"));
        patch(no_match, current_line());
    }

    emit_simple(Opcode::Pop, 0, "discard
unmatched selector");
    for (std::size_t jump : end_jumps) {
        patch(jump, current_line());
    }
}

```

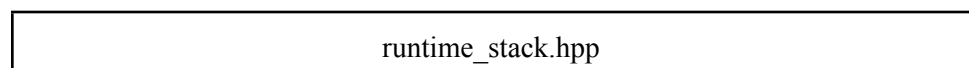
6. Output Built-in

Built-in write dan writeln diterjemahkan menjadi operasi output pada OPR. Setiap argumen dievaluasi ke stack terlebih dahulu. Untuk writeln, argumen terakhir menggunakan OPR WRTLN agar newline ditambahkan pada output akhir.



7. Runtime Stack

Runtime stack menyimpan operand sementara dan stack frame eksekusi. Setiap frame memiliki base, jumlah slot, static link, dynamic link, dan return address. Slot awal frame dipakai sebagai header, sedangkan slot berikutnya dipakai untuk parameter dan variabel lokal. Operasi LOD dan STO membaca serta menulis nilai berdasarkan lexical level dan address yang sudah diberikan oleh metadata semantic.



```

struct StackFrame {
    std::size_t base = 0;
    std::size_t slot_count = 0;
    int static_link = 0;
    int dynamic_link = 0;
    int return_address = 0;
};

class RuntimeStack {
public:
    explicit RuntimeStack(std::size_t
max_frames = 1000);
    bool push_frame(std::size_t
frame_slot_count,
                    std::size_t
parameter_slot_count,
                    int static_link,
                    int dynamic_link,
                    int return_address,
                    std::string* error =
                    nullptr);
    bool pop_frame(std::string* error =
                    nullptr);
    bool load(int level_delta, int offset,
RuntimeValue* out, std::string* error =
                    nullptr) const;
    bool store(int level_delta, int offset,
RuntimeValue value, std::string* error
                    = nullptr);
};

```

8. Interpreter

Interpreter menjalankan instruksi dengan siklus fetch-decode-execute. Instruction pointer menunjuk baris yang sedang dieksekusi, lalu opcode diproses melalui switch. Instruksi normal menaikkan IP ke baris berikutnya, sedangkan JMP, JPC, CAL, dan RET dapat mengubah IP.

interpreter.cpp

```

InterpreterResult
Interpreter::execute(const
std::vector<Instruction>& code) {
    InterpreterResult result;
    if (code.empty()) {

        result.diagnostics.push_back("Interpre
ter skipped: intermediate code is
empty.");
        return result;
    }
    RuntimeStack stack(options.max_frames);
    std::size_t ip = 0;
    PendingCall pending_call;

    while (true) {
        if (ip >= code.size()) {
            add_diagnostic(&result, ip,
            "Instruction pointer is out of
            bounds.");
            return result;
        }

        const Instruction& instruction
        = code[ip];
        const std::size_t current_ip = ip;
        ip = current_ip + 1;

        switch (instruction.opcode) {
            ...
        }
    }
}

```

9. Error Handling

Error handling dilakukan pada dua sisi. Loader DAST menghasilkan diagnostic untuk input yang bukan Decorated AST, node tidak dikenal, atau struktur pohon yang tidak valid. Interpreter menghasilkan diagnostic runtime untuk target jump tidak valid, stack error, frame error, dan division by zero. Dengan ini, program berhenti terkontrol dan menulis bagian error, bukan crash.

decorated_ast_loader.cpp

```

if (!parse_ast_kind(kind_name, &kind))
{
    diagnostics->push_back("Line " +
        std::to_string(line_number) +
        ": unknown
AST node kind '" + kind_name + "'");
    return nullptr;
}
if (looks_like_arion_source(stripped))
{
    result.diagnostics.push_back(
        "Milestone 4 input must be Decorated
AST, not Arion source code.");
    return result;
}

```

interpreter.cpp

```

bool validate_target(int target,
                    std::size_t
code_size,
                    std::size_t ip,
                    InterpreterResult*
result) {
    if (target >= 0 &&
        static_cast<std::size_t>(target) <
code_size) {
        return true;
    }
    std::ostringstream out;
    out << "Invalid jump target "
    << target << ".";
    add_diagnostic(result, ip, out.str());
    return false;
}

case OprCode::Div:
    if (is_zero(right)) {
        add_diagnostic(result, ip, "Division by
zero.");
        return false;
    }
}

```

III. Pengujian

Test Case / Input / Output
Test Case 1 : Menguji dasar backend: alokasi frame, assignment variabel, readback, dan writeln.
<pre>test > milestone-4 > ≡ tc_d1.txt 1 Program(M4Tc1BasicStoreLoad), tab_index=41 2 -- DeclarationPart 3 -- VarDecl(x) : type=Integer, tab_index=42 4 -- TypeRef(integer) : type=Integer, tab_index=32 5 ✓ -- CompoundStatement 6 -- AssignStatement : type=Integer, tab_index=42 7 -- Variable(x) : type=Integer, tab_index=42 8 -- Literal(10) value=10 literal=Integer : type=Integer 9 -- Call(writeln), tab_index=40 10 -- Variable(x) : type=Integer, tab_index=42 11 -- EmptyStatement 12</pre>
<pre>test > milestone-4 > ≡ tc_d1.txt_OUTPUT.txt 1 10 2</pre>
Test Case 2: Menguji ekspresi aritmetika dengan prioritas operator (+, -, *, div, mod, unary minus).

```

test > milestone-4 > tc_d2.txt
1 Program(M4Tc2ArithmeticPrecedence), tab_index=41
2 |-- DeclarationPart
3 | |-- VarDecl(x) : type=Integer, tab_index=42
4 | | |-- TypeRef(integer) : type=Integer, tab_index=32
5 | |-- VarDecl(y) : type=Integer, tab_index=43
6 | | |-- TypeRef(integer) : type=Integer, tab_index=32
7 |-- CompoundStatement
8 | |-- AssignStatement : type=Integer, tab_index=42
9 | | |-- Variable(x) : type=Integer, tab_index=42
10 | | |-- BinaryOp op=plus : type=Integer
11 | | | |-- BinaryOp op=minus : type=Integer
12 | | | | |-- BinaryOp op=plus : type=Integer
13 | | | | |-- Literal(2) value=2 literal=Integer : type=Integer
14 | | | | |-- BinaryOp op=times : type=Integer
15 | | | | |-- Literal(3) value=3 literal=Integer : type=Integer
16 | | | | |-- Literal(4) value=4 literal=Integer : type=Integer
17 | | | |-- BinaryOp op=idiv : type=Integer
18 | | | | |-- Literal(5) value=5 literal=Integer : type=Integer
19 | | | | |-- Literal(2) value=2 literal=Integer : type=Integer
20 | | | |-- BinaryOp op=imod : type=Integer
21 | | | | |-- Literal(7) value=7 literal=Integer : type=Integer
22 | | | | |-- Literal(3) value=3 literal=Integer : type=Integer
23 | |-- AssignStatement : type=Integer, tab_index=43
24 | | |-- Variable(y) : type=Integer, tab_index=43
25 | | |-- BinaryOp op=plus : type=Integer
26 | | | |-- UnaryOp op=minus : type=Integer, tab_index=42
27 | | | | |-- Variable(x) : type=Integer, tab_index=42
28 | | | | |-- Literal(20) value=20 literal=Integer : type=Integer
29 |-- Call(writeln), tab_index=40
30 | |-- Variable(x) : type=Integer, tab_index=42
31 |-- Call(writeln), tab_index=40
32 | |-- Variable(y) : type=Integer, tab_index=43
33 |-- EmptyStatement
34

```

```

test > milestone-4 > tc_d2.txt_OUTPUT.txt

```

```

1 13
2 7
3

```

Test Case 3: Menguji writeln multi-tipe (char, string, boolean) dan multi-argumen.


```
test > milestone-4 > tc_d3.txt
1 Program(M4Tc3TypedWrite), tab_index=41
2 |-- DeclarationPart
3 |   |-- VarDecl(flag) : type=Boolean, tab_index=42
4 |     |-- TypeRef(boolean) : type=Boolean, tab_index=35
5 |-- CompoundStatement
6 |   |-- AssignStatement : type=Boolean, tab_index=42
7 |     |-- Variable(flag) : type=Boolean, tab_index=42
8 |     |-- Variable(true) : type=Boolean, tab_index=37
9 |   |-- Call(writeln), tab_index=40
10 |     |-- Literal('A') value='A' literal=Char : type=Char
11 |     |-- Literal(' ') value=' ' literal=Char : type=Char
12 |     |-- Literal('ok') value='ok' literal=String : type=String
13 |   |-- Call(writeln), tab_index=40
14 |     |-- Variable(flag) : type=Boolean, tab_index=42
15 |-- EmptyStatement
16
```

```
test > milestone-4 > tc_d3.txt_OUTPUT.txt
```

```
1 A ok
2 true
3
```

Test Case 4: Menguji if-else dan alur jump (JPC, JMP).

test > milestone-4 > tc_d4.txt

```
1 Program(M4Tc4IfElseBranches), tab_index=41
2 |-- DeclarationPart
3 | |-- VarDecl(x) : type=Integer, tab_index=42
4 | |  |-- TypeRef(integer) : type=Integer, tab_index=32
5 | |-- VarDecl(y) : type=Integer, tab_index=43
6 | |  |-- TypeRef(integer) : type=Integer, tab_index=32
7 |-- CompoundStatement
8 | |-- AssignStatement : type=Integer, tab_index=42
9 | |  |-- Variable(x) : type=Integer, tab_index=42
10 | |    |-- Literal(8) value=8 literal=Integer : type=Integer
11 | |  |-- IfStatement : type=Boolean
12 | |    |-- BinaryOp op=gtr : type=Boolean
13 | |      |-- Variable(x) : type=Integer, tab_index=42
14 | |      |-- Literal(5) value=5 literal=Integer : type=Integer
15 | |      |-- AssignStatement : type=Integer, tab_index=43
16 | |        |-- Variable(y) : type=Integer, tab_index=43
17 | |        |-- Literal(1) value=1 literal=Integer : type=Integer
18 | |      |-- AssignStatement : type=Integer, tab_index=43
19 | |        |-- Variable(y) : type=Integer, tab_index=43
20 | |        |-- Literal(0) value=0 literal=Integer : type=Integer
21 | |  |-- Call(writeln), tab_index=40
22 | |    |-- Variable(y) : type=Integer, tab_index=43
23 | |  |-- AssignStatement : type=Integer, tab_index=42
24 | |    |-- Variable(x) : type=Integer, tab_index=42
25 | |    |-- Literal(2) value=2 literal=Integer : type=Integer
26 | |  |-- IfStatement : type=Boolean
27 | |    |-- BinaryOp op=gtr : type=Boolean
28 | |      |-- Variable(x) : type=Integer, tab_index=42
29 | |      |-- Literal(5) value=5 literal=Integer : type=Integer
30 | |      |-- AssignStatement : type=Integer, tab_index=43
31 | |        |-- Variable(y) : type=Integer, tab_index=43
32 | |        |-- Literal(9) value=9 literal=Integer : type=Integer
33 | |      |-- AssignStatement : type=Integer, tab_index=43
34 | |        |-- Variable(y) : type=Integer, tab_index=43
35 | |        |-- Literal(4) value=4 literal=Integer : type=Integer
36 | |  |-- Call(writeln), tab_index=40
37 | |    |-- Variable(y) : type=Integer, tab_index=43
38 | |  |-- EmptyStatement
39
```

test > milestone-4 > tc_d4.txt_OUTPUT.txt

```
1 1
2 4
3
```

Test Case 5: Menguji while, termasuk zero-iteration dan loop normal.

```
test > milestone-4 > tc_d5.txt
1  Program(M4Tc5WhileLoopEdges), tab_index=41
2  |-- DeclarationPart
3  |   |-- VarDecl(i) : type=Integer, tab_index=42
4  |   |-- TypeRef(integer) : type=Integer, tab_index=32
5  |-- CompoundStatement
6  |   |-- AssignStatement : type=Integer, tab_index=42
7  |   |   |-- Variable(i) : type=Integer, tab_index=42
8  |   |   |-- Literal(3) value=3 literal=Integer : type=Integer
9  |   |-- WhileStatement : type=Boolean
10 |   |   |-- BinaryOp op=lss : type=Boolean
11 |   |   |   |-- Variable(i) : type=Integer, tab_index=42
12 |   |   |   |-- Literal(3) value=3 literal=Integer : type=Integer
13 |   |   |-- CompoundStatement
14 |   |   |   |-- Call(writeln), tab_index=40
15 |   |   |   |-- Literal(99) value=99 literal=Integer : type=Integer
16 |   |   |   |-- EmptyStatement
17 |   |-- Call(writeln), tab_index=40
18 |   |-- Literal('after-zero-iteration') value='after-zero-iteration' literal=String : type=String
19 |-- AssignStatement : type=Integer, tab_index=42
20 |   |-- Variable(i) : type=Integer, tab_index=42
21 |   |-- Literal(0) value=0 literal=Integer : type=Integer
22 |-- WhileStatement : type=Boolean
23 |   |-- BinaryOp op=lss : type=Boolean
24 |   |   |-- Variable(i) : type=Integer, tab_index=42
25 |   |   |-- Literal(3) value=3 literal=Integer : type=Integer
26 |   |-- CompoundStatement
27 |   |   |-- Call(writeln), tab_index=40
28 |   |   |   |-- Variable(i) : type=Integer, tab_index=42
29 |   |   |-- AssignStatement : type=Integer, tab_index=42
30 |   |   |   |-- Variable(i) : type=Integer, tab_index=42
31 |   |   |   |-- BinaryOp op=plus : type=Integer
32 |   |   |   |   |-- Variable(i) : type=Integer, tab_index=42
33 |   |   |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
34 |   |   |   |-- EmptyStatement
35 |-- EmptyStatement
36
```

```
test > milestone-4 > tc_d5.txt_OUTPUT.txt
```

```
1  after-zero-iteration
2  0
3  1
4  2
5
```

Test Case 6: Menguji repeat-until (minimal satu eksekusi dan backward jump).

```

test > milestone-4 > ≡ tc_d6.txt
1  Program(M4Tc6RepeatUntil), tab_index=41
2  |-- DeclarationPart
3  |   |-- VarDecl(total) : type=Integer, tab_index=42
4  |   |-- TypeRef(integer) : type=Integer, tab_index=32
5  |-- CompoundStatement
6  |   |-- AssignStatement : type=Integer, tab_index=42
7  |   |   |-- Variable(total) : type=Integer, tab_index=42
8  |   |   |-- Literal(3) value=3 literal=Integer : type=Integer
9  |   |-- RepeatStatement : type=Boolean
10 |   |   |-- CompoundStatement
11 |   |   |   |-- Call(writeln), tab_index=40
12 |   |   |   |-- Variable(total) : type=Integer, tab_index=42
13 |   |   |   |-- AssignStatement : type=Integer, tab_index=42
14 |   |   |   |   |-- Variable(total) : type=Integer, tab_index=42
15 |   |   |   |   |-- BinaryOp op=minus : type=Integer
16 |   |   |   |   |   |-- Variable(total) : type=Integer, tab_index=42
17 |   |   |   |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
18 |   |   |   |-- EmptyStatement
19 |   |-- BinaryOp op=eql : type=Boolean
20 |   |   |-- Variable(total) : type=Integer, tab_index=42
21 |   |   |-- Literal(0) value=0 literal=Integer : type=Integer
22 |-- EmptyStatement
23

```

```

test > milestone-4 > ≡ tc_d6.txt_OUTPUT.txt

```

```

1  3
2  2
3  1
4

```

Test Case 7: Menguji for loop to dan downto serta update iterator.

```

test > milestone-4 > tc_d7.txt
1 Program(M4Tc7ForToDownto), tab_index=41
2 |-- DeclarationPart
3 | |-- VarDecl(i) : type=Integer, tab_index=42
4 | | \-- TypeRef(integer) : type=Integer, tab_index=32
5 | \-- VarDecl(total) : type=Integer, tab_index=43
6 | \-- TypeRef(integer) : type=Integer, tab_index=32
7 |-- CompoundStatement
8 | |-- AssignStatement : type=Integer, tab_index=43
9 | | |-- Variable(total) : type=Integer, tab_index=43
10 | | \-- Literal(0) value=0 literal=Integer : type=Integer
11 | |-- ForStatement(i) op=tosy, tab_index=42
12 | | |-- Literal(1) value=1 literal=Integer : type=Integer
13 | | |-- Literal(3) value=3 literal=Integer : type=Integer
14 | | \-- CompoundStatement
15 | | |-- AssignStatement : type=Integer, tab_index=43
16 | | | |-- Variable(total) : type=Integer, tab_index=43
17 | | | \-- BinaryOp op=plus : type=Integer
18 | | | |-- Variable(total) : type=Integer, tab_index=43
19 | | | \-- Variable(i) : type=Integer, tab_index=42
20 | | \-- EmptyStatement
21 | |-- Call(writeln), tab_index=40
22 | | \-- Variable(total) : type=Integer, tab_index=43
23 | |-- ForStatement(i) op=downtosy, tab_index=42
24 | | |-- Literal(3) value=3 literal=Integer : type=Integer
25 | | |-- Literal(1) value=1 literal=Integer : type=Integer
26 | | \-- CompoundStatement
27 | | |-- AssignStatement : type=Integer, tab_index=43
28 | | | |-- Variable(total) : type=Integer, tab_index=43
29 | | | \-- BinaryOp op=plus : type=Integer
30 | | | |-- Variable(total) : type=Integer, tab_index=43
31 | | | \-- Variable(i) : type=Integer, tab_index=42
32 | | \-- EmptyStatement
33 | |-- Call(writeln), tab_index=40
34 | | \-- Variable(total) : type=Integer, tab_index=43
35 |-- EmptyStatement
36

```

```

test > milestone-4 > tc_d7.txt_OUTPUT.txt

```

```

1 6
2 12
3

```

Test Case 8: Menguji boolean (and, or, not) dengan short-circuit.

```

test > milestone-4 > tc_d8.txt
1 Program(M4Tc8BooleanShortCircuit), tab_index=41
2 |-- DeclarationPart
3 | |-- VarDecl(zero) : type=Integer, tab_index=42
4 | |   \-- TypeRef(integer) : type=Integer, tab_index=32
5 | |-- VarDecl(flag) : type=Boolean, tab_index=43
6 | |   \-- TypeRef(boolean) : type=Boolean, tab_index=35
7 | \-- CompoundStatement
8 | |-- AssignStatement : type=Integer, tab_index=42
9 | | |-- Variable(zero) : type=Integer, tab_index=42
10 | | |   \-- Literal(0) value=0 literal=Integer : type=Integer
11 | |-- AssignStatement : type=Boolean, tab_index=43
12 | | |-- Variable(flag) : type=Boolean, tab_index=43
13 | | |   \-- Variable(false) : type=Boolean, tab_index=38
14 | |-- IfStatement : type=Boolean
15 | | |-- BinaryOp op=andsy : type=Boolean
16 | | | |-- Variable(flag) : type=Boolean, tab_index=43
17 | | | |   \-- BinaryOp op=eql : type=Boolean
18 | | | | |-- BinaryOp op=idiv : type=Integer
19 | | | | |   \-- Literal(10) value=10 literal=Integer : type=Integer
20 | | | | |   \-- Variable(zero) : type=Integer, tab_index=42
21 | | | | |   \-- Literal(0) value=0 literal=Integer : type=Integer
22 | | | |-- Call(writeln), tab_index=40
23 | | | | |   \-- Literal('and-bad') value='and-bad' literal=String : type=String
24 | | | |-- Call(writeln), tab_index=40
25 | | | | |   \-- Literal('and-ok') value='and-ok' literal=String : type=String
26 | |-- AssignStatement : type=Boolean, tab_index=43
27 | | |-- Variable(flag) : type=Boolean, tab_index=43
28 | | |-- Variable(true) : type=Boolean, tab_index=37
29 | |-- IfStatement : type=Boolean
30 | | |-- BinaryOp op=orsy : type=Boolean
31 | | | |-- Variable(flag) : type=Boolean, tab_index=43
32 | | | |   \-- BinaryOp op=eql : type=Boolean
33 | | | | |-- BinaryOp op=idiv : type=Integer
34 | | | | |   \-- Literal(10) value=10 literal=Integer : type=Integer
35 | | | | |   \-- Variable(zero) : type=Integer, tab_index=42
36 | | | | |   \-- Literal(0) value=0 literal=Integer : type=Integer
37 | | | |-- Call(writeln), tab_index=40
38 | | | | |   \-- Literal('or-ok') value='or-ok' literal=String : type=String
39 | | | |-- Call(writeln), tab_index=40
40 | | | | |   \-- Literal('or-bad') value='or-bad' literal=String : type=String
41 | |-- IfStatement : type=Boolean
42 | | |-- UnaryOp op=notsy : type=Boolean, tab_index=38
43 | | | |-- Variable(false) : type=Boolean, tab_index=38
44 | | | |-- Call(writeln), tab_index=40
45 | | | | |   \-- Literal('not-ok') value='not-ok' literal=String : type=String
46 | |-- EmptyStatement
47

```

```

test > milestone-4 > tc_d8.txt_OUTPUT.txt

```

```

1 and-ok
2 or-ok
3 not-ok
4

```

Test Case 9: Menguji operasi real, termasuk arithmetic dan branching.

```
test > milestone-4 > tc_d9.txt
1 Program(M4Tc9RealArithmetic), tab_index=41
2 |-- DeclarationPart
3 | \-- VarDecl(r) : type=Real, tab_index=42
4 | \-- TypeRef(real) : type=Real, tab_index=33
5 \-- CompoundStatement
6 | \-- AssignStatement : type=Real, tab_index=42
7 | | \-- Variable(r) : type=Real, tab_index=42
8 | | \-- BinaryOp op=plus : type=Real
9 | | | \-- UnaryOp op=minus : type=Real
10 | | | \-- Literal(-2.5) value=-2.5 literal=Real : type=Real
11 | | \-- BinaryOp op=rdiv : type=Real
12 | | | \-- Literal(6.0) value=6.0 literal=Real : type=Real
13 | | | \-- Literal(2.0) value=2.0 literal=Real : type=Real
14 | \-- Call(writeln), tab_index=40
15 | | \-- Variable(r) : type=Real, tab_index=42
16 | \-- IfStatement : type=Boolean
17 | | \-- BinaryOp op=geq : type=Boolean
18 | | | \-- Variable(r) : type=Real, tab_index=42
19 | | | \-- Literal(0.5) value=0.5 literal=Real : type=Real
20 | | \-- Call(writeln), tab_index=40
21 | | | \-- Literal('real-ok') value='real-ok' literal=String : type=String
22 | | \-- Call(writeln), tab_index=40
23 | | | \-- Literal('real-bad') value='real-bad' literal=String : type=String
24 | \-- EmptyStatement
25
```

```
test > milestone-4 > tc_d9.txt_OUTPUT.txt
```

```
1 0.5
2 real-ok
3
```

Test Case 10: Menguji fitur kompleks (array, record, procedure, function, case).

```

test > milestone-4 > tc_d10.bit
1  Program(M4Tc10CompositeCallCaseCoverage), tab_index=41
2  |-- DeclarationPart
3  |   |-- TypeDecl(Index) : type=Index, tab_index=42
4  |   |   \-- RangeType : type=Index
5  |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
6  |   |   \-- Literal(2) value=2 literal=Integer : type=Integer
7  |   |-- TypeDecl(Values) : type=Values, tab_index=43
8  |   |   \-- ArrayType : type=Values
9  |   |   |-- TypeRef(Index) : type=Index, tab_index=42
10 |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
11 |   |-- TypeDecl(Box) : type=Box, tab_index=45
12 |   |   \-- RecordType : type=Box
13 |   |   |-- FieldDecl(value) : type=Integer, tab_index=44
14 |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
15 |   |-- VarDecl(values) : type=Values, tab_index=46
16 |   |   \-- TypeRef(Values) : type=Values, tab_index=43
17 |   |-- VarDecl(box) : type=Box, tab_index=47
18 |   |   \-- TypeRef(Box) : type=Box, tab_index=45
19 |   |-- VarDecl(x) : type=Integer, tab_index=48
20 |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
21 |   |-- ProcedureDecl(SetX), tab_index=49, block=2
22 |   |   |-- ParameterGroup
23 |   |   |   |-- VarDecl(value) : type=Integer, tab_index=50, lev=1
24 |   |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
25 |   |   |-- DeclarationPart
26 |   |   |   \-- CompoundStatement, lev=1, block=2
27 |   |   |       |-- AssignStatement : type=Integer, tab_index=48
28 |   |   |       |   |-- Variable(x) : type=Integer, tab_index=48, lev=1
29 |   |   |       |   \-- Variable(value) : type=Integer, tab_index=50, lev=1
30 |   |   |       \-- EmptyStatement
31 |   |   \-- FunctionDecl(Twice) : type=Integer, tab_index=51, block=3
32 |   |       |-- TypeRef(integer) : type=Integer, tab_index=32
33 |   |       |-- ParameterGroup
34 |   |       |   |-- VarDecl(value) : type=Integer, tab_index=52, lev=1
35 |   |       |   \-- TypeRef(integer) : type=Integer, tab_index=32
36 |   |       |-- DeclarationPart
37 |   |       |   \-- CompoundStatement, lev=1, block=3
38 |   |       |       |-- AssignStatement : type=Integer, tab_index=51
39 |   |       |       |   |-- Variable(Twice) : type=Integer, tab_index=51, lev=1
40 |   |       |       |   \-- BinaryOp op=times : type=Integer
41 |   |       |       |       |-- Variable(value) : type=Integer, tab_index=52, lev=1
42 |   |       |       |       \-- Literal(2) value=2 literal=Integer : type=Integer
43 |   |       |       \-- EmptyStatement
44 |   |   \-- CompoundStatement
45 |   |       |-- AssignStatement : type=Integer, tab_index=46
46 |   |       |   |-- Variable(values) : type=Integer, tab_index=46
47 |   |       |   |   \-- IndexComponent
48 |   |       |   |       \-- Literal(1) value=1 literal=Integer : type=Integer
49 |   |       |   \-- Literal(7) value=7 literal=Integer : type=Integer
50 |   |       |-- AssignStatement : type=Integer, tab_index=44
51 |   |       |   |-- Variable(box) : type=Integer, tab_index=44
52 |   |       |   |   \-- FieldComponent(value)
53 |   |       |   \-- Variable(values) : type=Integer, tab_index=46
54 |   |       |       \-- IndexComponent
55 |   |       |           \-- Literal(1) value=1 literal=Integer : type=Integer
56 |   |       |-- Call(SetX), tab_index=49
57 |   |       |   \-- Variable(box) : type=Integer, tab_index=44
58 |   |       |       \-- FieldComponent(value)
59 |   |       |-- AssignStatement : type=Integer, tab_index=48
60 |   |       |   |-- Variable(x) : type=Integer, tab_index=48
61 |   |       |   \-- Call(Twice) : type=Integer, tab_index=51
62 |   |       |       \-- Variable(x) : type=Integer, tab_index=48
63 |   |       |-- CaseStatement : type=Integer
64 |   |       |   |-- Variable(x) : type=Integer, tab_index=48
65 |   |       |   |-- CaseBranch
66 |   |       |   |   |-- Literal(14) value=14 literal=Integer : type=Integer
67 |   |       |   |   \-- Call(writeln), tab_index=40
68 |   |       |   |       \-- Literal('fourteen') value='fourteen' literal=String : type=String
69 |   |       |   \-- CaseBranch
70 |   |       |       |-- Literal(0) value=0 literal=Integer : type=Integer
71 |   |       |       \-- Call(writeln), tab_index=40
72 |   |       |           \-- Literal('zero') value='zero' literal=String : type=String
73 |   |   \-- EmptyStatement
74

```



```
test > milestone-4 > ≡ tc_d10.txt_OUTPUT.txt
1  fourteen
2
```

Test Case 11: Negative test AST invalid (variabel tanpa deklarasi). Harus ditolak sebelum VM.

```
test > milestone-4 > ≡ tc_d11.txt
1  Program(M4MalformedMissingSymbolInfo)
2  \-- CompoundStatement
3      |-- AssignStatement
4      |   |-- Variable(x)
5      |   \-- Literal(10) value=10 literal=Integer
6      |-- Call(writeln)
7      |   \-- Variable(x)
8      \-- EmptyStatement
9
```

```
test > milestone-4 > ≡ tc_d11.txt_OUTPUT.txt
1  === BACKEND INPUT ERRORS ===
2  4:1: Error: Semantic error: Identifier 'x' is not declared
3  7:1: Error: Semantic error: Identifier 'x' is not declared
4
```

IV. Kesimpulan

Implementasi backend berhasil menerjemahkan Decorated AST menjadi instruksi stack machine seperti INT, LIT, LOD, STO, JMP, JPC, OPR, dan RET. Selain instruksi utama tersebut, beberapa instruksi tambahan seperti LDA, LDI, STI, CHK, DUP, dan POP digunakan untuk mendukung kebutuhan implementasi seperti akses alamat, array, record, dan pengelolaan stack pada control flow.

Decorated AST loader menjadi komponen penting karena menjembatani format input M4 dengan struktur internal compiler. Loader membaca format tree dari Decorated AST, kemudian SemanticAnalyzer digunakan kembali untuk melakukan rehydration metadata seperti symbol table, type registry, lexical level, tab index, block index, dan address variabel. Dengan pendekatan ini, backend tidak bergantung pada metadata yang dibuat secara hardcoded, tetapi tetap memakai struktur semantik yang sama dengan pipeline compiler sebelumnya.

Interpreter dan runtime stack memungkinkan Intermediate Code yang dihasilkan benar-benar dieksekusi. Output akhir program, terutama hasil write dan writeln, ditulis pada bagian `== OUTPUT =`, sedangkan instruksi yang dijalankan ditulis pada bagian `= INTERMEDIATE CODE ==`. Format output ini dibuat minimal agar sesuai dengan kebutuhan Milestone 4, yaitu hanya menampilkan Intermediate Code dan output aktual program.

Secara keseluruhan, Milestone 4 memperluas Arion dari compiler yang hanya melakukan analisis hingga menjadi sistem yang dapat menghasilkan dan menjalankan representasi antara. Implementasi ini juga menjaga kompatibilitas dengan milestone sebelumnya karena pipeline lexical, syntax, AST, dan semantic untuk source code tetap tersedia dan tidak dihapus.

V. Saran

Untuk pengembangan berikutnya, format Decorated AST sebagai input backend sebaiknya dibuat semakin resmi melalui serializer dan deserializer khusus. Dengan format yang lebih terstruktur, misalnya berbasis section yang konsisten atau format data terstandar, proses loading DAST dapat menjadi lebih tahan terhadap variasi whitespace, anotasi tambahan, dan perubahan kecil pada printer semantic.

VI. Lampiran

A. Link Repository Github

<https://github.com/alfarabi0642/PRX-Tubes-IF2224-2026>

B. Pembagian Tugas

NIM	Tugas	Persentase Kontribusi
13524002	integrasi backend, output convention, README, dokumentasi akhir.	25%
13524022	code generator dari DAST, ekspresi, statement, control flow, address mapping, jump patching.	25%
13524086	baseline backend, Makefile, runtime stack, interpreter, diagnostic runtime.	25%
13524094	regression test M1-M4 dan dokumentasi pengujian.	25%

C. Flowchart

Flowchart dapat dilihat pada link berikut:

 [FLOWCHART TBFO M4](#)

VII. Referensi

1. Tutorialspoint. "Compiler Design - Lexical Analysis."
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.html
2. J. E. Hopcroft, R. Motwani, dan J. D. Ullman. Introduction to Automata Theory, Languages, and Computation, 3rd ed.
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf
3. A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. (Dragon Book), 2006.
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)